HEAT EQUATION FOR 2D BI-MATERIAL PLATE.

Advanced Computational Engineering Project.

Abstract

Investigating the heat transfer dynamics within a two-dimensional plate composed of two distinct materials in equal proportions, subject to specified boundary conditions at the plate ends. The governing equations for heat transfer have been determined. A sophisticated computational engineering code has been employed to solve these equations, with subsequent refinement of the results.

Contribution Statement

The concept behind the calculation was initial proposed by Smita and perfected and finalized by all 3 of us. The serial and the parallel code were written by Smita with a bit of edits made to the parallel code with Devansh. Vignesh applied OpenMP to both the codes. Testing of the codes on Artemis was done by Devansh. The report was made by Devansh.

Devansh Pratap Kakkar - 530664451

Shubhrasmita Mandal - 530158949

Vignesh Kubendran - 530550202

Introduction:

The Problem statement:

We are solving an heat diffusion problem, with set parameters on the boundary conditions. We have taken a solid plate (No fluid motion), made up of 2 different material (dividing it into equal halves, with a line perpendicular on the x-axis) with heat diffusion coefficients, $\alpha_1$and α_2. We are going to solve the heat diffusion problem in two dimensions, using an iterative solving method over time step of 0.02 second in each iteration over 100 iterations. The main objectives of this study is to gain insight in using numerical methods to solve governing equations, getting used to solving heat transfer equations using coding and learn to critically analyse and plot acquired result data from said code. We can verify our result by verifying our code, validating our mathematical methodologies and verifying our solution generated. We have divided the plate up to 200×200 equal pieces, with 200 division along the x-axis and 200 along y-axis.

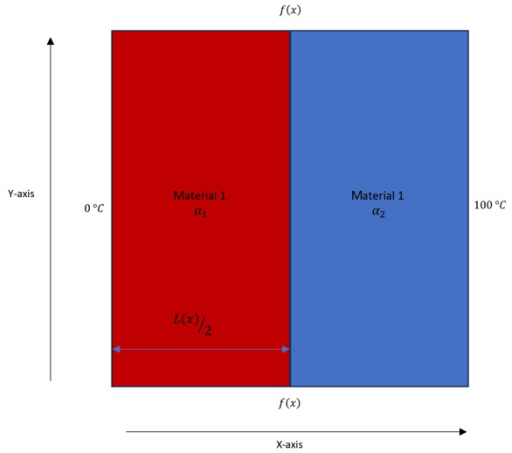

Figure 1: Pictorial representation of Problem statement.

Significance of the Problem:

The usage of numerical methods in the field of heat transfer are experiencing an substantial growth, both in industry and research. But solving these problems for a multitude of different cases, can be highly energy and time consuming. We are doing this study to find the most effective way to solve heat transfer problems, using time, energy and money in the most efficient way. This can be doing by using, better numerical methods, using efficient coding methods (like using MPI, OpenMP), or using a clean coding method which doesn't use computing resources unnecessarily (lesser if statements, no unnecessary printing, etc).

Governing Equations and Assumptions:

To solve the problem, we have made the following assumption:

1. The plate is made of solid materials (No fluid motion).
2. The temperature is fixed to $0°C$ on the left corner and to $100°C$ on the right corner. This does not mean the maximum temperature that the plate can have is $100°C$. These are called the **Dirichlet boundary conditions**.

$$T = 0°C \; \forall \; x = 0 \; ; t = 0 \; sec$$
$$T = 100°C \; \forall \; x = L(x) \; ; t = 0 \; sec \tag{1}$$

3. It is a 100% efficient heat transfer system, hence there is no loss of energy in any form.
4. We are assuming the diffusivity constants of both the materials to be equal at $t = 0\ sec$ for code simplicity.

The governing equations for the initial boundary conditions on the top and bottom boundary of the plate is given in equation (2):

$$f(x) = \frac{100 \times x}{L(x)} + sin^2\left(\frac{\pi \times y}{L(y)}\right) \forall\ 0 \leq x \leq L(x)\ ; t = 0\ sec \tag{2}$$

The governing equations for heat dissipation inside the boundary of the 2D plate is given in equation (3) , (4) and (5) :

$$\frac{\partial T}{\partial t} = \alpha_1 \times \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) \forall\ 0 < x < {}^{L(x)}/_{2}\ ;\ 0 < x < L(y) \tag{3}$$

$$\frac{\partial T}{\partial t} = \alpha_2 \times \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) \forall\ {}^{L(x)}/_{2} < x < L(x)\ ;\ 0 < x < L(y) \tag{4}$$

$$\frac{\partial T}{\partial t} = \alpha_{12} \times \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}\right) \forall\ x = {}^{L(x)}/_{2}\ ;\ 0 < x < L(y)\ ; \alpha_{12} = \frac{\alpha_1 + \alpha_2}{2} \tag{5}$$

Numerical Method:

To solve the governing equations, we are using the Jacobian solver method and making insights into the changes in grid size and how that will affect the convergence of the solution.

We are taking a partial differential in both space and time, the following differentials can be expanded as such,

$$\frac{\partial T}{\partial t} = \frac{T(n+1) - T(n)}{\Delta t} \tag{6}$$

We are taking a first order forward finite differential with time, this is because, even though a central differential formula is more accurate, it can yield zero derivatives which can ruin the result, especially if the domain of the derivative is discrete. [1]

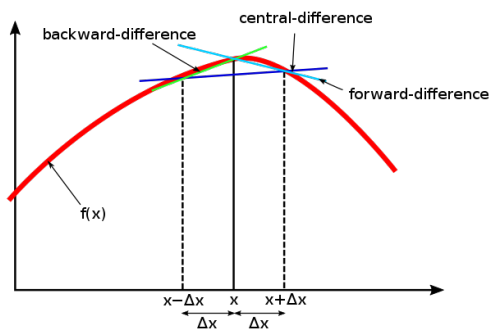

Figure 2: First order Finite differential. [1]

$$\frac{\partial^2 T}{\partial x^2} = \frac{T(x+1) - 2 \times T(x) + T(x-1)}{\Delta x^2} \tag{7}$$

$$\frac{\partial^2 T}{\partial y^2} = \frac{T(y+1) - 2 \times T(y) + T(y-1)}{\Delta y^2} \tag{8}$$

We are taking a second order central finite differential with space. This is because temperature with space can take any form of value, so it's a continuous variable.

Inputting the values from equation 6, 7 and 8 into equation 3, 4 and 5, we get the following partial differential equations:

$$\frac{T(n+1) - T(n)}{\Delta t} = \alpha_1 \times \left(\frac{T(x+1) - 2 \times T(x) + T(x-1)}{\Delta x^2} + \frac{T(y+1) - 2 \times T(y) + T(y-1)}{\Delta y^2} \right) \forall\, 0 < x < {}^{L(x)}/_{2}\ ;\ 0 < x < L(y) \tag{9}$$

$$\frac{T(n+1) - T(n)}{\Delta t} = \alpha_2 \times \left(\frac{T(x+1) - 2 \times T(x) + T(x-1)}{\Delta x^2} + \frac{T(y+1) - 2 \times T(y) + T(y-1)}{\Delta y^2} \right) \forall\, {}^{L(x)}/_{2} < x < L(x)\ ;\ 0 < x < L(y) \tag{10}$$

$$\frac{T(n+1) - T(n)}{\Delta t} = \alpha_{12} \times \left(\frac{T(x+1) - 2 \times T(x) + T(x-1)}{\Delta x^2} + \frac{T(y+1) - 2 \times T(y) + T(y-1)}{\Delta y^2} \right) \forall\, x = {}^{L(x)}/_{2}\ ;\ 0 < x < L(y)\ ; \alpha_{12} = \frac{\alpha_1 + \alpha_2}{2} \tag{11}$$

Solving equation 9,10 and 11 together to get a single general equation of Temperature for the whole domain (The 2D plate is called a domain, which is converted into a i,j matrix with i in the x direction, denoting the number of columns and j is in the y direction, denoting number of rows. This is because Fortran is a column specific language) given below:

$$T_{i,j}^{n+1} = \left(\frac{\alpha_{T_1} + \alpha_{T_2}}{2}\right) \times \frac{\Delta t}{\Delta x^2} \times \left(T_{i,j+1}^{n} - T_{i,j}^{n}\right) - \left(\frac{\alpha_{T_1} + \alpha_{T_2}}{2}\right) \times \frac{\Delta t}{\Delta x^2} \times \left(T_{i,j}^{n} - T_{i,j-1}^{n}\right) + \alpha_T \times \frac{\Delta t}{\Delta y^2} \times \left(T_{i+1,j}^{n} - 2 \times T_{i,j}^{n} + T_{i-1,j}^{n}\right) + T_{i,j}^{n} \quad (12)$$

Here, α_{T_1} and α_{T_2} represent the Diffusivity factor of the domain location who's temperature is being considered in the X direction. In the X direction, the diffusivity factor changes after crossing L(x)/2. If we are considering a location behind L(x)/2, the value of $\left(\frac{\alpha_{T_1}+\alpha_{T_2}}{2}\right)$ become α_1 and if we are ahead of L(x)/2, the value of $\left(\frac{\alpha_{T_1}+\alpha_{T_2}}{2}\right)$ become α_2. At L(x)/2, the value of $\left(\frac{\alpha_{T_1}+\alpha_{T_2}}{2}\right)$ become α_{12}.
α_T on the contrary varies in the Y directions, so it remains the same for the particular domain location, i.e. it doesn't change with moving up or down on the domain.
For example, if we consider the temperature of a point x,y lying on x = L(x)/2, on the domain at time = n and try to find the temperature after time Δt, at time = n+1. Equation 12 becomes as shown below:

$$T_{i,j}^{n+1} = \left(\frac{\alpha_2 + \alpha_{12}}{2}\right) \times \frac{\Delta t}{\Delta x^2} \times \left(T_{i,j+1}^{n} - T_{i,j}^{n}\right) - \left(\frac{\alpha_{12} + \alpha_1}{2}\right) \times \frac{\Delta t}{\Delta x^2} \times \left(T_{i,j}^{n} - T_{i,j-1}^{n}\right) + \alpha_{12} \times \frac{\Delta t}{\Delta y^2} \times \left(T_{i+1,j}^{n} - 2 \times T_{i,j}^{n} + T_{i-1,j}^{n}\right) + T_{i,j}^{n} \quad (13)$$

Equation 13 is made, particularly for easy coding. The pictorial representation of how the Jacobian method is used to discretise and solve the heat plate problem is given below:

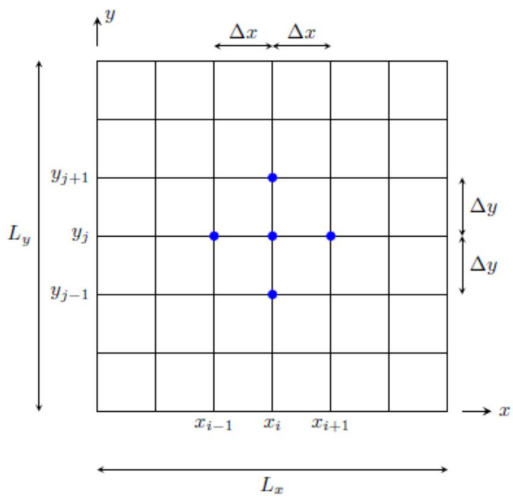

Figure 3: Jacobian Solver.

Program:

We have solved the following code using three complex methodologies, being a serial code, an serial OpenMP code and lastly gone a step further with an Parallel OpenMP code (Hybrid Code). We are hoping to find significant changes in time taken to compile the code, as the

coding method becomes more and more parallel. As we are using the Jacobian method to do iterative analysis, the code is highly parallelizable.

Serial code:

The series code is fairly simple. We have used nested Do loops to solve the temperature values being iterated along the whole domain at a single time step. And a do while loop outside to iterate it for the whole 2 seconds. We have also introduced a stability criteria function to insure that the solver is stable across the whole domain. A Tec plot file is printed for the whole domain at each time step.

Serial OpenMP code:

The serial code is parallelised using OpenMP. The part of the code which has been parallelised are the Initialising alpha values for the whole domain, the stability criteria for the whole domain, the nested do loops for the calculation of temperature for the whole domain in a single time step and the printing of the Tec plot file. The tec plot file is only printed for 3 time steps, which are 0 second, 1 second and 2 second.

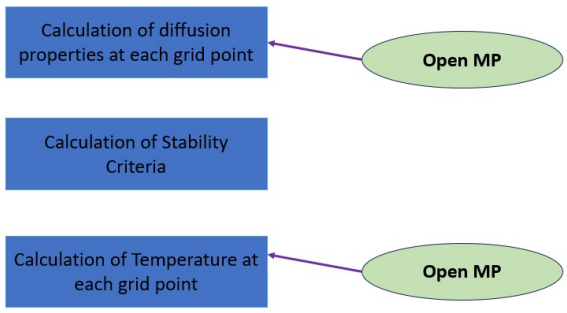

Figure 4: Serial code using OpenMP.

Parallel OpenMP code (Hybrid code):

When using parallel coding with OpenMP, it is safe to introduce OpenMP inside parallel code, and not the other way around. This is because OpenMP only exchange variables in a shared memory space, so if we use MPI inside OpenMP, the different threads will not be able to share data.

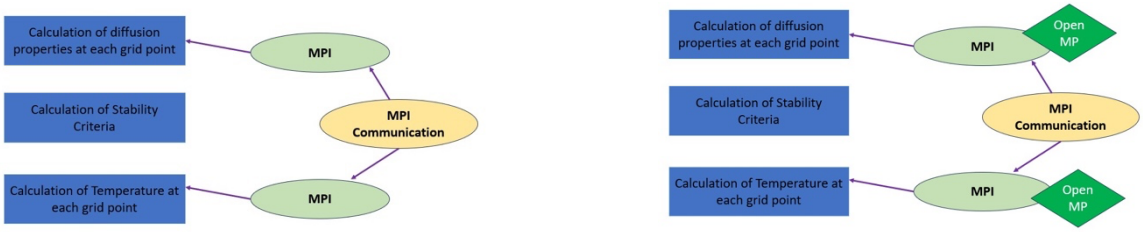

Figure 5:
Hybrid code using MPI and OpenMP.

We are introducing OpenMP between each Processor to increase the parallelisation. As it can be seen in figure 5, the temperature calculation of the whole plate between each iteration is divided using MPI. Inside every PID, we are dividing the work between Threads using OpenMP. Then all the values are communicated using MPI.

Input and output:

We have already made the programs in a manner such that it does not require any inputs. The output is a tec plot file which will be visualised to see the temperature values all over the domain. We are also calculating the run time taken to solve the whole domain.

User guide:

The user can copy the code to their favourite coding platform and run serial and serial OpenMP code using Fortran commands. The Hybrid code can be run using MPI commands in the Linus or Cygwin command terminal.

Result:

The result is generated using Artemis. We are running only the OpenMP code to generated the results. We are running the code over 6 different grid values to compare the data and the speed increase/decrease as the grid size increases. We are also changing the number of cores (Threads in OpenMP) used to iterate a solution to see the change in performance as the number of cores increase. We are using 20 cores at max to solve the equation.

Validation of Result:

The result generated on the Tec plot can be seen below:

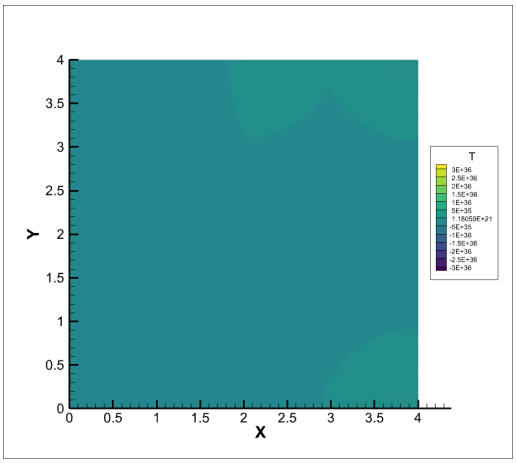

Figure 6 : Plot of Temperature over the plate at time = 0 sec.

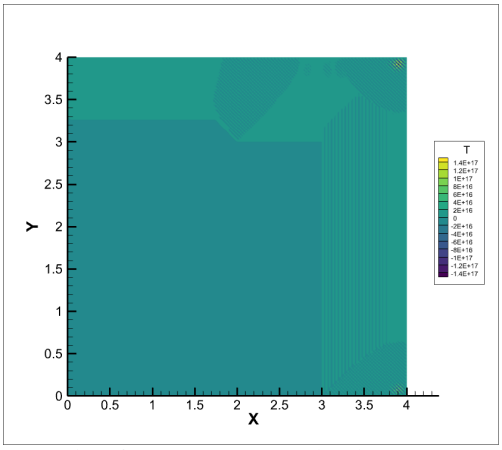

Figure 7: Plot of Temperature over the plate at time = 1 sec.

As it can be seen in figure 6, the temperature on the top and bottom boundary increases as x value increases. This is in line with the initial F(x) function for the boundary condition.

As the time passes, the temperature can be seen mixing up (seen in figure 7) as the plate starts exchanging temperature. It is worth noticing that the temperature can be seen moving leftwards as the initial temperature is maximum at the right boundary and minimum at left boundary.

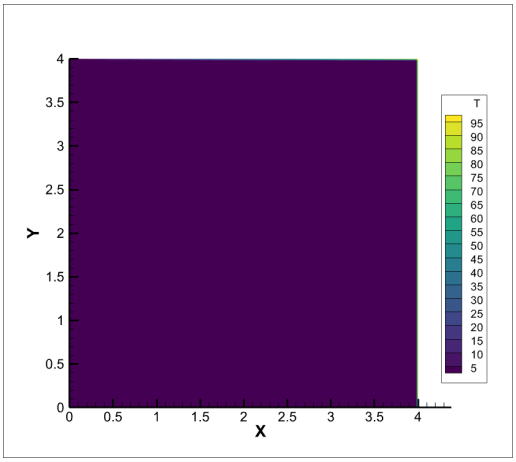

Figure 8: Plot of Temperature over the plate at time = 2 sec.

After the iteration is stopped, the temperature averages out over the whole plate. The boundary conditions still remain the same (Seen in figure 8).

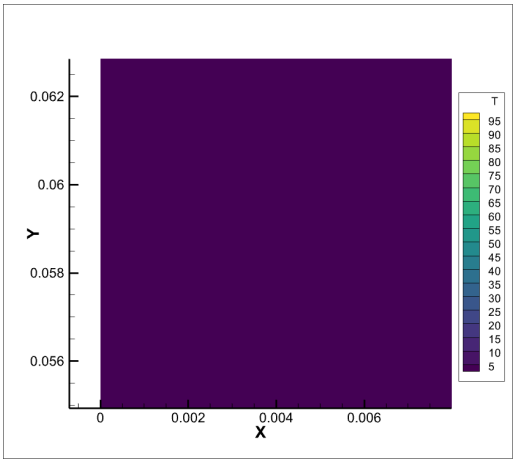

Figure 9: Left boundary of the plate at time = 2 sec.

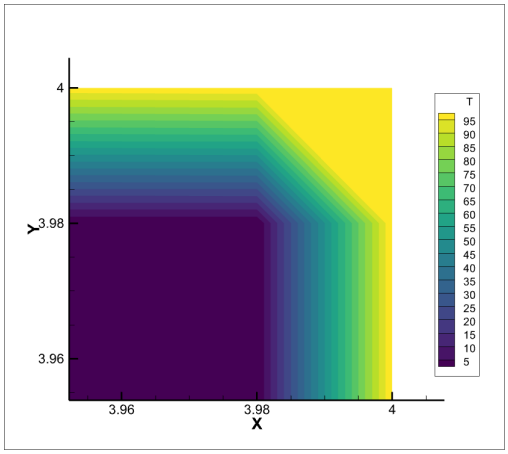

Figure 10: Top right corner of the plate at time = 2 sec.

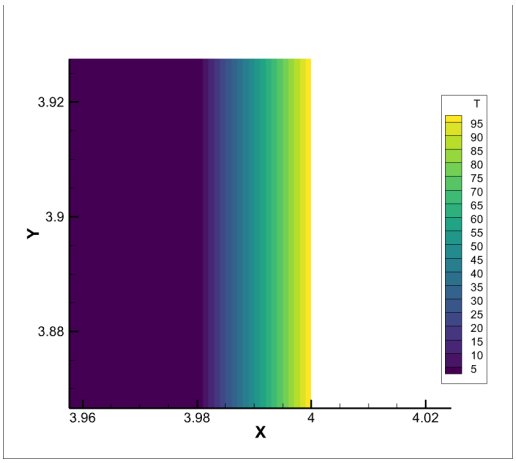

Figure 11: Right boundary of the plate at time = 2 sec.

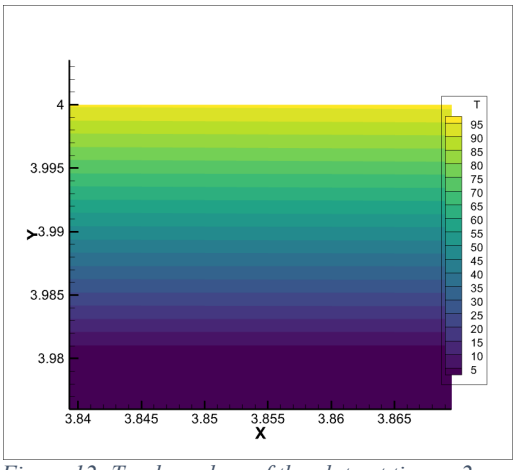

Figure 12: Top boundary of the plate at time = 2 sec.

The boundary values can be seen from figure 8 to figure 12. All these values generated are in terms with the governing equations and the boundary conditions given.

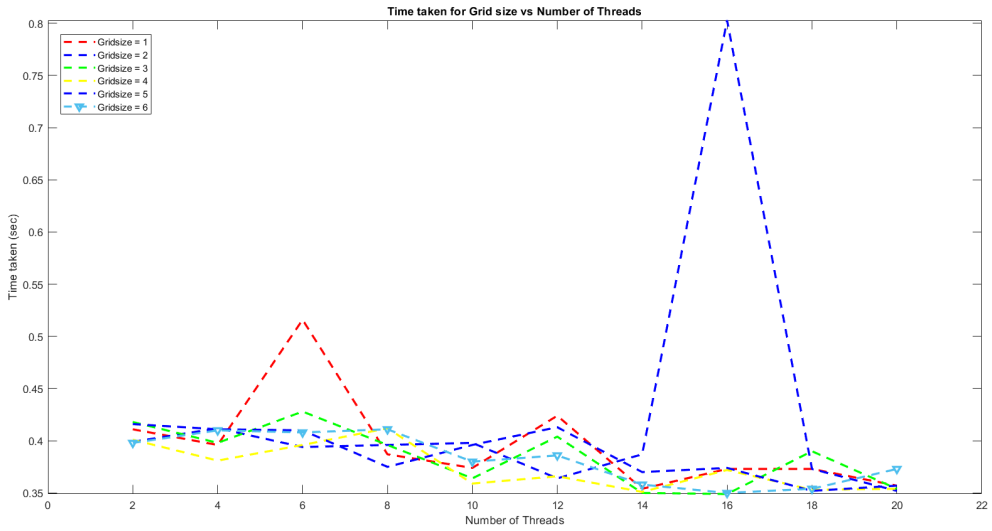

Figure 13: Plot of time vs Number of cores used.

It can be seen from the data generated, that as the number of grids increase, the time taken to compute the solution decreases. It is also clear as the number of threads increase, the average time taken to compute the solution seems to be having a negative slope for each grid. This is in terms with the expected solution.

Methods of Efficiency Employed:

The methods used to improve efficiency is to change the grid size which has an active effect on the amount of time taken to solve the domain. But there is a downside to this change, as the grid becomes courser and courser, the error in each calculation increases.

There is another method to increase efficiency, and that is to parallelise the code using MPI or OpenMP. This method does not save money, but it decreases the time used to do the calculations. This is done by increasing the number of cores (Threads in OpenMP) used to calculate the result.

Bibliography

[1] “Wikipedia,” [Online]. Available: https://en.wikipedia.org/wiki/Finite_difference.